

Assignment: Multi-Window Media Sequencer with Sync Playback

Build and deploy a full-stack application where multiple display windows continuously play media in their own configured sequence. The system must support dynamic playlist changes and a sync action that shows one selected media item across all windows at the same time.

Core Scenario

Each window has its own media list. The total play size for each window must be treated as 5 hours. Each window should keep playing its configured list again and again within that cycle. Blank is only a configured playlist item when included; the rest of the cycle should not become blank playback by default.

Expected Sync Behavior

When a user triggers sync for a media item, for example M2, every window should display M2 at the same time. After the configured sync duration ends, each window should continue its own normal sequence without losing its playlist configuration.

Frontend Requirements

- Build the frontend using React.
- Display multiple media windows on the screen.
- Continuously play each window's assigned media list in sequence without stopping between items.
- Support image media, video media, and a blank or fallback state.
- Reflect playlist changes from the backend.
- Provide controls to add media to a window's list and trigger sync playback.

Backend Requirements

- Build the backend using Golang.
- Store media lists in any persistent storage of the candidate's choice.
- Implement logic to dynamically add media to each window's list.
- Implement sync logic so one selected media item can appear in all windows at the same time.
- The implementation approach is flexible. The backend does not need to expose an API that continuously serves the next media item, as long as the system reliably displays media continuously and handles sync correctly.
- No suggested API structure is required. Candidates may design their own API contracts.

Deployment Requirement

- Deploy the React frontend.
- Deploy the Golang backend.
- Provide live URLs or clear access instructions.
- Document deployment steps and environment or configuration requirements.

Expected Deliverables

- React frontend application.
- Golang backend application.
- Persistent data storage implementation.
- Seed data matching the example windows and media lists.
- README with setup instructions, sync behavior explanation, deployment instructions, assumptions, and API documentation if APIs are exposed.

Evaluation Criteria

- Correct continuous playback per window.
- Correct 5-hour cycle behavior where each window restarts its media list after the sequence ends.
- Ability to dynamically update media lists.
- Reliable sync behavior across all windows.
- Clean React and Golang code structure.
- Persistent storage usage.
- Successful frontend and backend deployment.

Candidate Notes

Candidates may choose their preferred database, hosting provider, and implementation architecture. The final submission should clearly explain assumptions, tradeoffs, and how the sync behavior was implemented.